

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who not less than 40 marks in Subject 1:** Count the number of

Operation...

Submit

```
1 import numpy as np
2
3 a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
4
5 # 1. Print all student details
6 print("All student Details:\n",a)
7
8 # 2. print total students
9
10 print("Total Students:",a.shape[0] )
11
```

Average time0.014 s14.00 ms

Maximum time0.014 s14.00 ms

1 out of 1 shown test case(s) passed

Test case 114 msDebug

Expected output

Actual output

All student Details:	All student Details:
[[301. 67. 77. 88.]	[[301. 67. 77. 88.]
[302. 78. 88. 77.]	[302. 78. 88. 77.]
[303. 45. 56. 89.]	[303. 45. 56. 89.]
[304. 88. 98. 45.]	[304. 88. 98. 45.]
[305. 78. 88. 99.]	[305. 78. 88. 99.]
[306. 68. 78. 36.]	[306. 68. 78. 36.]

TerminalTest cases

3.1.1. Numpy Array Operations

05:25



Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, r and c , representing the number of rows and the number of columns.
- The next r lines contain c space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases



numpyarr...

Submit

```
1 import numpy as np
2
3 # write your code here...
4 row,column = map(int,input().split())
5
6 elements = []
7
8 for _ in range(row):
9     elements.extend(map(int,input().split()))
10
11 array = np.array(elements).reshape(row,column)
12
13 print(array)
14 print(array.ndim)
15 print(array.shape)
16 print(array.size)
```

Average time

0.007 s

7.42 ms

Maximum time

0.011 s

11.00 ms

2 out of 2 shown test case(s) passed

10 out of 10 hidden test case(s) passed

Test case 1 11 ms

Debug

Expected output

3 4

1 2 3 4

Actual output

3 4

1 2 3 4

Terminal

Test cases

< Prev

Reset

Submit

Next >

3.2.1. Numpy: Matrix Operations

08:52

The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

Task:
You are required to compute and display the results of the following matrix operations:

1. **Addition** (`matrix_a + matrix_b`)
2. **Subtraction** (`matrix_a - matrix_b`)
3. **Element-wise Multiplication** (`matrix_a * matrix_b`)
4. **Matrix Multiplication** (`matrix_a . matrix_b`)
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:
The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

matrixOp...

```
5 matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Matrix B:")
8 matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
9
10
11 # Addition
12 madd=matrix_a + matrix_b
13 print("Addition (A + B):")
14 print(madd)
15 # Subtraction
```

Average time
0.013 s
12.75 ms

Maximum time
0.016 s
16.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 16 ms Debug

Expected output	Actual output
Enter Matrix A:	Enter Matrix A:
1 2 3	1 2 3
4 5 6	4 5 6
7 8 9	7 8 9
Enter Matrix B:	Enter Matrix B:
1 1 1	1 1 1

Terminal

Test cases

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

09:31

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

+

stacking.py

Submit

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9
10 # Perform horizontal stacking (hstack)
11 a = np.hstack((arr1, arr2))
```

Average time

0.013 s

12.75 ms

Maximum time

0.017 s

17.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 17 ms

Debug

Expected output

Enter Array1:

1 2 3

4 5 6

7 8 9

Enter Array2:

4 5 6

Actual output

Enter Array1:

1 2 3

4 5 6

7 8 9

Enter Array2:

4 5 6

Terminal

Test cases

< Prev

Reset

Submit

Next >

3.2.3. Numpy: Custom Sequence Generation

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Sample Test Cases

customS...

Submit

```
1 import numpy as np
2
3 # Take user input for the start, stop, and step of the sequence
4 start = int(input())
5 stop = int(input())
6 step = int(input())
7
8 # Generate the sequence using np.arange()
9
10 a = np.arange(start, stop, step)
11
```

Average time
0.006 s
6.50 ms

Maximum time
0.010 s
10.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 10 ms Debug

Expected output	Actual output
3	3
10	10
2	2
[3 5 7 9]	[3 5 7 9]

Test case 2 6 ms

Terminal

Test cases

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bit...

14.45



You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: $A_i \text{ OR } B_i$).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases



different...



Submit

```
1 import numpy as np
2
3 def array_operations(A, B):
4
5     # Convert A and B to NumPy arrays
6     array_a = np.array(A)
7
8     array_b = np.array(B)
9
10    # Arithmetic Operations
11    sum_result = array_a + array_b
```

Average time

0.007 s

6.67 ms



Maximum time

0.009 s

9.00 ms



1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

9 ms

Debug



Expected output

1 2 3 4

5 6 7 8

Actual output

1 2 3 4

5 6 7 8

Element-wise Sum: 6 8 10 12

Element-wise Sum: 6 8 10 12

Element-wise Difference: -4 -4 -4 -4

Element-wise Difference: -4 -4 -4 -4

Element-wise Product: 5 12 21 32

Element-wise Product: 5 12 21 32

Mean of A: 2.5

Mean of A: 2.5

Terminal

Test cases

< Prev

Reset

Submit

Next >

3.2.5. Numpy: Copying and Viewing Arrays

03:23



The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

Original array after modifying view: `<original_array>`View array: `<view_array>`

- After modifying the copy:

Original array after modifying copy: `<original_array>`Copy array: `<copy_array>`

Sample Test Cases



copyAnd...

Submit

```
1 import numpy as np
2
3 inputlist = list(map(int,input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputlist)
7
8 # Create a view
9 view_array = original_array.view()
10
11 # Create a copy
```

Average time

0.004 s

3.75 ms



Maximum time

0.007 s

7.00 ms



2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

7 ms

Debug



Expected output

10 20 30 40 50 60 70 80

Actual output

10 20 30 40 50 60 70 80

Original array after modifying view: [99 20 30 40 50 60 70 80]

Original array after modifying view: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Copy array: [10 88 30 40 50 60 70 80]

Copy array: [10 88 30 40 50 60 70 80]

Terminal

Test cases

< Prev

Reset

Submit

Next >

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

08:49



The given code in the editor takes a single array, array1, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. **Searching:** Find the indices where search_value appears in array1 and print these indices.
2. **Counting:** Count how many times count_value appears in array1 and print the count.
3. **Broadcasting:** Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
4. **Sorting:** Sort array1 in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.

Output Format:

1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.
4. The sorted array.

Explorer

arrayOpe...



Submit

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split()))))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11 # Find indices where value matches in array1
```

Average time

0.009 s

8.75 ms



Maximum time

0.012 s

12.00 ms



2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 12 ms

Debug



Expected output

1 1 1 2 2 2

Value to search: 1

Value to count: 2

Value to add: 2

[0 1 2]

3

Actual output

1 1 1 2 2 2

Value to search: 1

Value to count: 2

Value to add: 2

[0 1 2]

3

Terminal

Test cases

< Prev

Reset

Submit

Next >